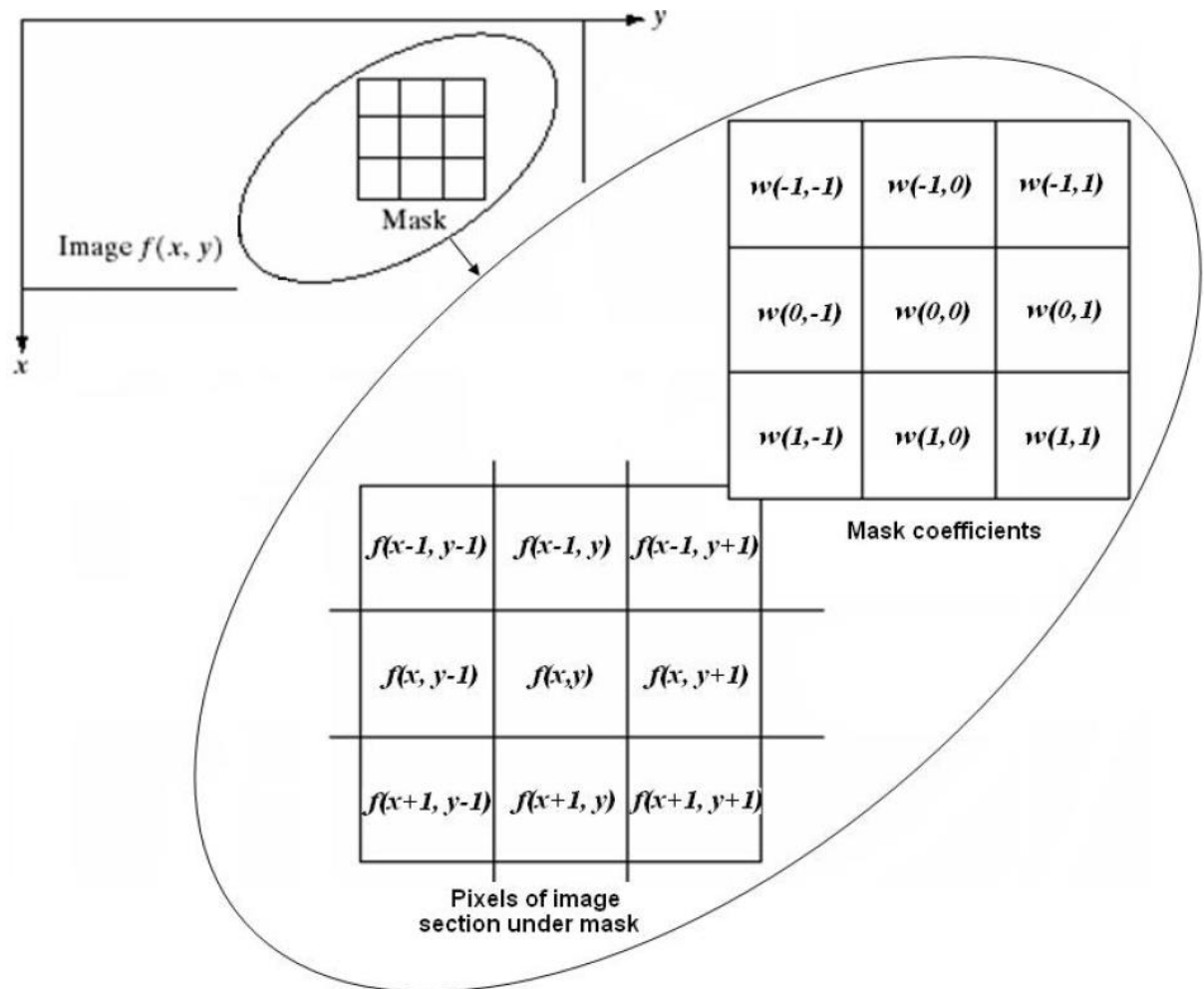


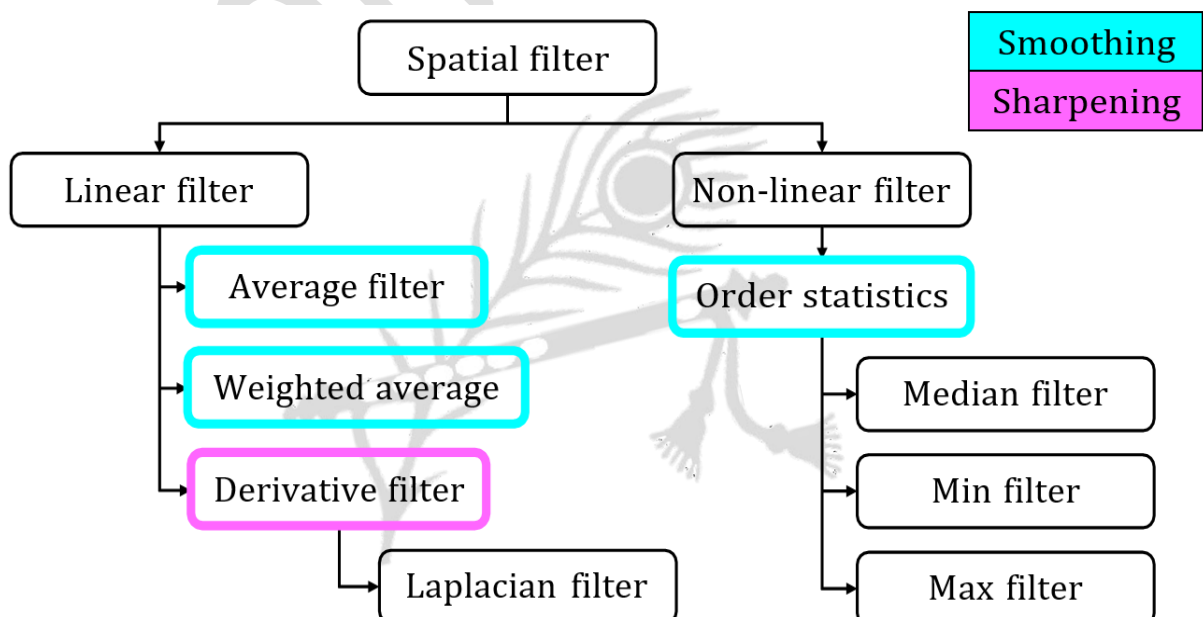
Spatial Filtering

- **Spatial Filtering** enhances images by modifying a pixel's intensity based on its neighbors, using predefined operations.
- It involves image operators that change the gray value at any pixel (x,y) based on the pixel values in a square neighborhood centered at (x,y) .
- A fixed integer matrix of the same size as the neighborhood of the original image is used for the filtering operation. This matrix is called a **filter, mask, kernel, template, or window**.
- The filter matrix's elements are called **coefficients** and represent weights applied during processing. These elements are not pixel intensities.
- Spatial filtering is a powerful tool for enhancing or suppressing specific image features, such as smoothing, sharpening, edge detection, and noise reduction.
- **The mechanism of spatial filtering** consists of moving the filter mask from pixel to pixel across the image. **The mask size must be odd (i.e. 3x3, 5x5, etc.),** ensuring it has a center.
- At each pixel (x,y) , the filter's response is calculated using a predefined relationship, which can be linear or nonlinear. In

the figure below, w is a filter mask, and $f(x,y)$ is an image.



➤ Spatial filter is categorized by linear and non-linear filter



- **Linear Filter:** At each pixel (x, y) , the filter response is calculated as the **sum of products** between the filter coefficients and the corresponding image pixels within the area covered by the filter mask.
- For the above image, the filter mask is 3×3 , and the response R after applying the mask is calculated as:

$$R = w(-1, -1)f(x-1, y-1) + w(-1, 0)f(x-1, y) + \dots + w(1, 1)f(x+1, y+1)$$

Here, $w(s, t)$ represents the filter coefficients, and $f(x+s, y+t)$ is the corresponding image pixels.

- For an image $f(x, y)$ of size $M \times N$ with a filter mask $w(s, t)$ of size $m \times n$, the filtered output $g(x, y)$ is:

$$g(x, y) = \sum_{s=-a}^a \sum_{t=-b}^b w(s, t) f(x + s, y + t)$$

$$\text{where } a = \frac{m-1}{2} \text{ and } b = \frac{n-1}{2}.$$

- To process the entire image, the equation is applied for all pixels, where $x = 0, 1, 2, \dots, M-1$ and $y = 0, 1, 2, \dots, N-1$.

Example:

			a	b	c	
			d	e	f	
			g	h	i	

Original Image $f(x,y)$

r	s	t
u	v	w
x	y	z

Filter Mask

Here e is the center pixel and a, b, c, d, f, g, h, i are the neighborhood of e . The 3×3 filter mask is convolved with this center pixel e which is given by

$$R_e = r*a + s*b + t*c + u*d + v*e + w*f + x*g + y*h + z*i$$

The above process is repeated for every pixel in the original image to generate the filtered image.

For the border pixel, we need to use zero padding or border padding techniques.

- **Zero Padding:** Zeros are added around the image border to extend it.

1	3	5
2	4	0
6	8	10

0	0	0	0	0
0	1	3	5	0
0	2	4	0	0
0	6	8	10	0
0	0	0	0	0

- **Border Padding:** The existing border pixels are replicated to extend the image.

1	3	5
2	4	0
6	8	10

1	1	3	5	5
1	1	3	5	5
2	2	4	0	0
6	6	8	10	10
6	6	8	10	10

Example:

Use the following 3×3 mask to perform the convolution process on the shaded pixels in the 5×5 image below. Write the filtered image.

0	$\frac{1}{6}$	0
$\frac{1}{6}$	$\frac{1}{3}$	$\frac{1}{6}$
0	$\frac{1}{6}$	0

 3×3 mask

30	40	50	70	90
40	50	80	60	100
35	255	70	0	120
30	45	80	100	130
40	50	90	125	140

 5×5 image

Solution:

$$0 \times 30 + \frac{1}{6} \times 40 + 0 \times 50 + \frac{1}{6} \times 40 + \frac{1}{3} \times 50 + \frac{1}{6} \times 80 + 0 \times 35 + \frac{1}{6} \times 255 + 0 \times 70 = 85$$

$$0 \times 40 + \frac{1}{6} \times 50 + 0 \times 70 + \frac{1}{6} \times 50 + \frac{1}{3} \times 80 + \frac{1}{6} \times 60 + 0 \times 255 + \frac{1}{6} \times 70 + 0 \times 0 = 65$$

$$0 \times 50 + \frac{1}{6} \times 70 + 0 \times 90 + \frac{1}{6} \times 80 + \frac{1}{3} \times 60 + \frac{1}{6} \times 100 + 0 \times 70 + \frac{1}{6} \times 0 + 0 \times 120 =$$

$$0 \times 40 + \frac{1}{6} \times 50 + 0 \times 80 + \frac{1}{6} \times 35 + \frac{1}{3} \times 255 + \frac{1}{6} \times 70 + 0 \times 30 + \frac{1}{6} \times 45 + 0 \times 80 = 118$$

and so on ...

Filtered image =

30	40	50	70	90
40	85	65	61	100
35	118	92	58	120
30	84	77	89	130
40	50	90	125	140

Nonlinear filters are particularly useful for tasks where linear filtering may struggle, such as noise reduction in the presence of non-Gaussian noise or preserving image edges while reducing noise.

Characteristics of Nonlinear Filters:

1. **Pixel-wise Conditional Operations:** Nonlinear filters apply operations conditionally based on the pixel values within a neighborhood, unlike linear filters that rely on predefined coefficients for convolution.
2. **Non-convolutional Nature:** They do not use a sum-of-products framework, which is central to linear filters.
3. **Preservation of Non-linear Features:** These filters maintain image details and edges better in comparison to linear filters. Noise reduction in images with speckle or impulsive noise.

Example: Median Filter

Smoothing Spatial Filters

- Used for **blurring** and **noise reduction**.
- Removes small details from an image before extracting larger objects.
- Bridges small gaps in lines or curves.
- **Noise Reduction** is achieved by applying blurring using linear filters or nonlinear filters.

Average Filter:

The average filter (or weighted average filter) is used to smooth an $M \times N$ image by replacing each pixel's value with

the average (or weighted average) of its neighboring pixel intensities.

- It works by convolving the image with a mask (filter) of size $m \times n$.
- The output pixel value is the mean of the pixels covered by the filter mask.
- The general mathematical expression for filtering is:

$$g(x, y) = \frac{\sum_{s=-a}^a \sum_{t=-b}^b w(s, t) f(x + s, y + t)}{\sum_{s=-a}^a \sum_{t=-b}^b w(s, t)}$$

- Average filter is also called Low pass filter
- **Applications:**
 - Useful for smoothing an image.
 - Highlights coarse or gross details while suppressing fine details and noise.

A 3x3 Average Filter mask:

$$\frac{1}{9} \begin{array}{|c|c|c|} \hline 1 & 1 & 1 \\ \hline 1 & 1 & 1 \\ \hline 1 & 1 & 1 \\ \hline \end{array}$$

For 1x3 filter mask = $1/3[1 \ 1 \ 1]$

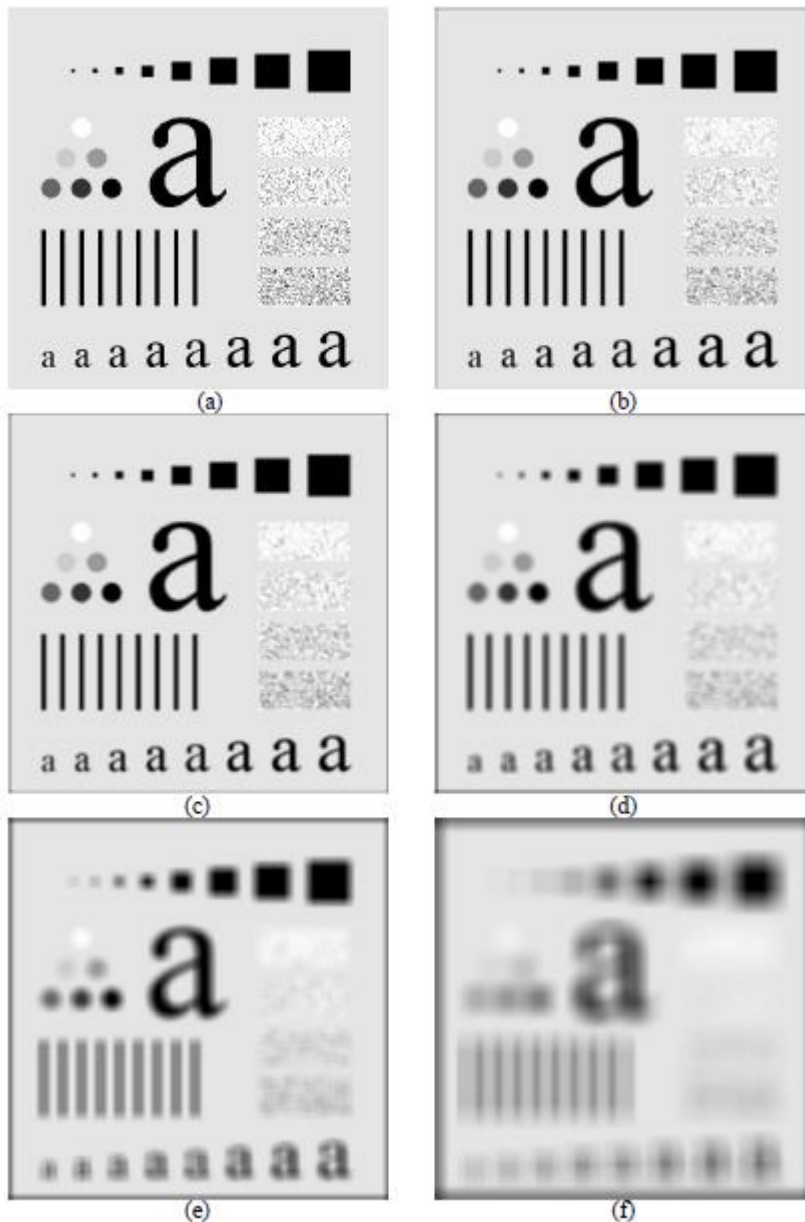


Fig: Effect of average filter (a) Original image of 500 x 500 pixels (b) Results of smoothing with averaging filter masks of size $m=n=3, 5, 9, 15,$ and $35,$ respectively.

Note: big mask is used to eliminate small objects from an image.

Weighted average filter has more different coefficients to give more importance to some pixels at the expense of others. The idea behind this is to reduce the blurring effect in the smoothing process. Pixels closer to the central pixel are more important. A 3x3 mask of a weighted average filter is given by

$$\frac{1}{16}$$

1	2	1
2	4	2
1	2	1

End Term 2024

Consider an image as shown in Fig. 5 (alias I_6). Suppose we apply a 1×3 average filter on an image I_6 to get the image I_7 . Again, we apply the same filter on image I_7 to get the image I_8 .

2	5	6	6	7	4	3
---	---	---	---	---	---	---

Fig. 5

Find the final value of the center pixel.

3

Solution: $I_6 = [2, 5, 6, 6, 7, 4, 3]$

Pad Zero, $I_7 = [0, 2, 5, 6, 6, 7, 4, 3, 0]$

1×3 average filter, $\frac{1}{3}[1 \ 1 \ 1]$

Consider pixel 2 as a center pixel and apply the filter.

$$\frac{1}{3}(0 \times 1 + 2 \times 1 + 5 \times 1) = \frac{1}{3}(7) = 2.33 = 2$$

Shift the filter on the image.

$$\text{For center pixel 5, } \frac{1}{3}(2 \times 1 + 5 \times 1 + 6 \times 1) = 4.33 = 4$$

$$\text{For center pixel 6, } \frac{1}{3}(5 \times 1 + 6 \times 1 + 6 \times 1) = 5.66 = 6$$

$$\text{For center pixel 6, } \frac{1}{3}(6 \times 1 + 6 \times 1 + 7 \times 1) = 6.33 = 6$$

$$\text{For center pixel 7, } \frac{1}{3}(6 \times 1 + 7 \times 1 + 4 \times 1) = 5.67 = 6$$

$$\text{For center pixel 4, } \frac{1}{3}(7 \times 1 + 4 \times 1 + 3 \times 1) = 4.67 = 5$$

$$\text{For center pixel 3, } \frac{1}{3}(4 \times 1 + 3 \times 1 + 0 \times 1) = 2.33 = 2$$

$$I_7=[2,4,6,6,6,5,2]$$

$$\text{Pad zero again, } I_8=[0,2,4,6,6,6,5,2,0]$$

Consider pixel 2 as a center pixel and apply the filter.

$$1/3(0 \times 1 + 2 \times 1 + 4 \times 1) = 1/3(6) = 2$$

Shift the filter on the image I_8

$$\text{For center pixel 4, } 1/3(2 \times 1 + 4 \times 1 + 6 \times 1) = 4$$

$$\text{For center pixel 6, } 1/3(4 \times 1 + 6 \times 1 + 6 \times 1) = 5.33 = 5$$

$$\text{For center pixel 6, } 1/3(6 \times 1 + 6 \times 1 + 6 \times 1) = 6$$

$$\text{For center pixel 6, } 1/3(6 \times 1 + 6 \times 1 + 5 \times 1) = 5.67 = 6$$

$$\text{For center pixel 4, } 1/3(6 \times 1 + 5 \times 1 + 2 \times 1) = 4.33 = 4$$

$$\text{For center pixel 3, } 1/3(5 \times 1 + 2 \times 1 + 0 \times 1) = 2.33 = 2$$

$$I_8=[2,4,5,6,6,4,2]$$

Order-statistics filters

are nonlinear spatial filters whose response is based on ordering (ranking) the pixels contained in the image area encompassed by the filter.

- These are **nonlinear spatial filters** that operate by ranking (ordering) the pixel values within the neighborhood defined by the filter.
- The output of the filter is determined based on a specific rank or statistical property of the ordered pixels.

Key Characteristics:

- Common examples include:
 - **Median filter:** Outputs the median value of the sorted neighborhood, effective for reducing salt-and-pepper noise.
 - **Max filter:** Outputs the maximum pixel value, useful for enhancing bright regions.
 - **Min filter:** Outputs the minimum pixel value, helpful for enhancing dark regions.
- These filters preserve edges better compared to linear filters and are particularly effective in removing impulse noise while maintaining important image details.

Median Filter:

- The **median filter** is a nonlinear spatial filter used to reduce impulsive noise, such as **salt-and-pepper noise**, in images. It works by replacing the central pixel in a neighborhood with the **median value** of all intensities within that neighborhood.

Noise Reduction: Effectively removes salt-and-pepper noise, where "salt" represents high-intensity values (e.g., 255) and "pepper" represents low-intensity values (e.g., 0).

Salt-and-Pepper Noise:

- Appears as random black and white pixels in an image.
- Typically caused by errors in data transmission or sensor faults.
- Its PDF is bimodal with two sharp peaks.

Advantages:

- Preserves edges and details while reducing noise.
- Avoids the blurring effect of linear filters.
- Removes Salt and pepper noise.

Disadvantages:

- **Computationally Expensive:** Sorting pixel values is time-consuming for large images or windows.
- **Less Effective on Gaussian Noise:** Performs poorly compared to Gaussian or Wiener filters.
- **Over-Smoothing:** Can alter uniform areas unnecessarily.
- **Detail Loss:** Larger windows may blur fine details and textures.

Applications:

- Commonly used in **medical imaging**, **remote sensing**, and **computer vision** for preprocessing and improving image quality.

- In the $M \times N$ mask, the median is the middle element.

Q. Find the median value for the given matrix.

10	15	20
20	100	20
20	20	25

Solution:

- At first sorting the pixel values of the matrix.
- 10, 15, 20, 20, **20**, 20, 20, 25, 100

Example:

Consider the following 5×5 image:

20	30	50	80	100
30	20	80	100	110
25	255	70	0	120
30	30	80	100	130
40	50	90	125	140

Apply a 3×3 median filter on the shaded pixels, and write the filtered image.

Solution

20	30	50	80	100
30	20	80	100	110
25	255	70	0	120
30	30	80	100	130
40	50	90	125	140

Sort:

20, 25, 30, 30, 30, 70, 80, 80, 255

20	30	50	80	100
30	20	80	100	110
25	255	70	0	120
30	30	80	100	130
40	50	90	125	140

Sort

0, 20, 30, 70, 80, 80, 100, 100, 255

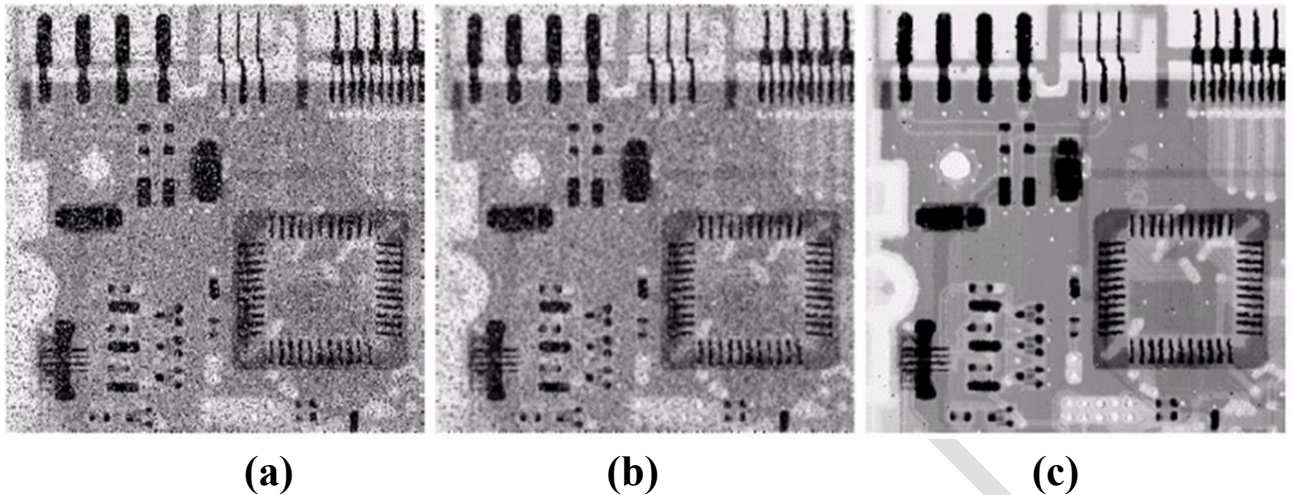
20	30	50	80	100
30	20	80	100	110
25	255	70	0	120
30	30	80	100	130
40	50	90	125	140

Sort

0, 70, 80, 80, 100, 100, 110, 120, 130

Filtered Image =

20	30	50	80	100
30	20	80	100	110
25	30	80	100	120
30	30	80	100	130
40	50	90	125	140

**Fig:**

- a) X-ray image of circuit board corrupted by salt-pepper noise
- b) Noise reduction with a 3 x 3 averaging mask
- c) Noise reduction with a 3 x 3 median filter

10	15	20
20	100	20
20	20	25

MAX Filter:

- Sort the values of pixels in the matrix.
- In the MxN mask, the maximum is the last element.
- 10, 15, 20, 20, 20, 20, 20, 25, **100**

MIN Filter: Minimum is the 1st element.

- **10**, 15, 20, 20, 20, 20, 20, 25, 100

Mid Term 2024

Consider the following image. What will be the new value of the pixel (2, 2) if smoothing is done using a 3x3:

- Mean Filter
- Min Filter
- Median Filter

3

240	11	100	93
124	178	229	56
20	42	190	0
5	77	230	60

Solution: The problem asks to calculate the new value of the pixel at position (2, 2) using a 3x3 neighborhood smoothing filter with different types: mean filter, min filter, and median filter.

The 3x3 neighborhood around pixel (2, 2) is: 190

$$\begin{bmatrix} 178 & 229 & 56 \\ 42 & 190 & 0 \\ 77 & 230 & 60 \end{bmatrix}$$

a) Mean Filter: To apply the mean filter, calculate the average of all the values in the 3x3.

$$\text{Mean} = 118$$

So, the new value for pixel (2, 2) using the mean filter is 118.

b) Min Filter: The min filter replaces the pixel value with the minimum value from the 3x3 neighborhood.

$\text{Min} = \min(0, 42, 56, 60, 77, 178, 190, 230, 229) = 0$

So, the new value for pixel (2, 2) using the min filter is 0.

c) Median Filter:

The median filter replaces the pixel value with the median of the 3x3 neighborhood.

First, arrange the pixel values in ascending order:

0, 42, 56, 60, 77, 178, 190, 230, 229

The median is the middle value (5th value): 77

So, the new value for pixel (2, 2) using the median filter is 77.

Sharpening Filter:

A **sharpening filter** is used to highlight fine details (e.g. edges) in an image or enhance detail in an image that has been blurred through errors or imperfect capturing devices. Since **blurring** is achieved by averaging filter (which is similar to integration), **sharpening** is done through **differentiation**, which enhances edges and discontinuities. That means a sharpening spatial filter inverts the averaging filter.

Key Points:

- **Objective:** Enhances edges and restores blurred details.
- **Uses differentiation:** Both first and second-order derivatives are used to detect intensity changes.

- **Applications:** Used in medical imaging, printing, industrial inspection, and military guidance systems.

First-order derivatives:

The first-order partial derivatives of a digital image $f(x, y)$ are:

$$\frac{\partial f}{\partial x} = f(x + 1, y) - f(x, y) \quad \text{and} \quad \frac{\partial f}{\partial y} = f(x, y + 1) - f(x, y)$$

- Response of first-order derivative is:
 - zero in flat segments
 - Non-zero at the onset (beginning) of a ramp
 - Non-zero along ramps

Second-order derivatives:

The second-order partial derivatives of a digital image $f(x, y)$ are:

$$\frac{\partial^2 f}{\partial x^2} = f(x + 1, y) + f(x - 1, y) - 2f(x, y)$$

$$\frac{\partial^2 f}{\partial y^2} = f(x, y + 1) + f(x, y - 1) - 2f(x, y)$$

- Response of second order derivative is:
 - Zero in flat areas
 - Non-zero at the onset (beginning) of a ramp
 - Zero along ramps of constant slope

Eg:

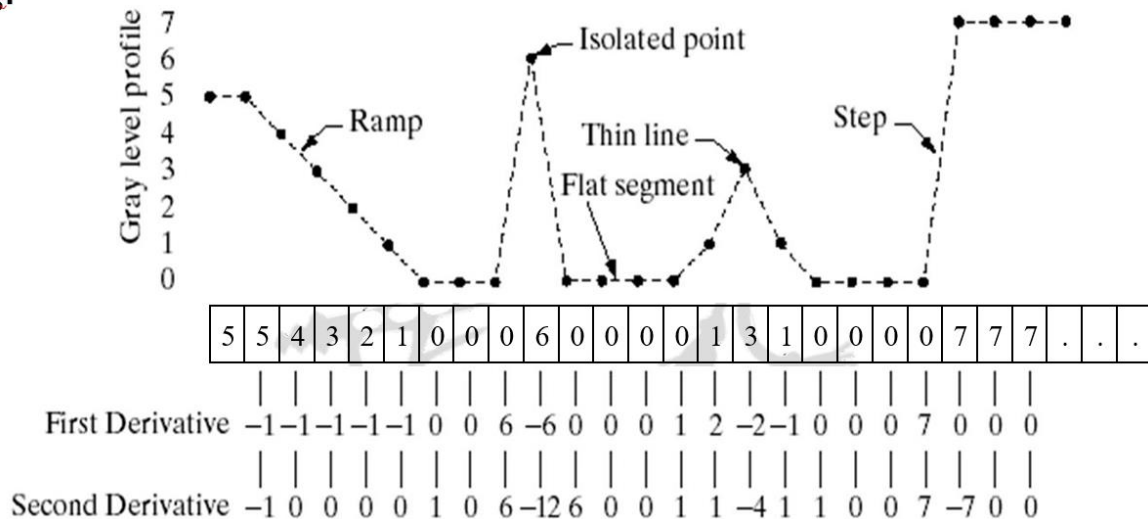


Fig: Example of partial derivatives

- First-order derivatives generally produce thicker edges in images.
- Second-order derivatives have a stronger response to fine detail (e.g. thin lines or isolated points).
- First-order derivatives respond more strongly to sudden changes in gray levels.
- Second-order derivatives produce two peaks at gray-level transitions.
- The filter should be isotropic, meaning its response remains the same regardless of the direction of edges in the image.

Laplacian filter:

The Laplacian operator of a digital image $f(x, y)$ is:

$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

$$\nabla^2 f = f(x+1, y) + f(x-1, y) + f(x, y+1) + f(x, y-1) - 4f(x, y)$$

Since the Laplacian filter is a linear spatial filter, we can apply it using the same mechanism of the convolution process. The 3x3 L_4 and L_8 Laplacian filter masks are given below:

90-degree isotropic filter:

0	1	0
1	-4	1
0	1	0

0	-1	0
-1	4	-1
0	-1	0

45-degree isotropic filter:

1	1	1
1	-8	1
1	1	1

-1	-1	-1
-1	8	-1
-1	-1	-1

Laplacian Filter Characteristics

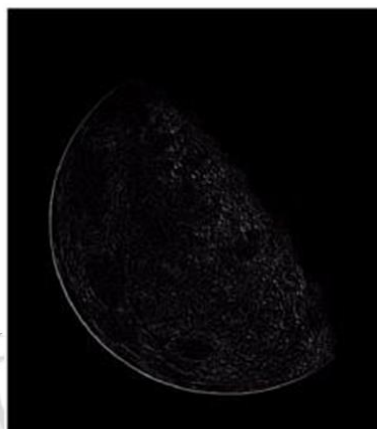
- As a **derivative operator**, the Laplacian filter enhances regions where there are **sudden changes in intensity** (edges) while suppressing areas with **gradual intensity variations**.
- It is particularly effective in highlighting **gray-level discontinuities** in an image.

Effects on the Image

- The Laplacian filter produces an output where **edges and sharp transitions** appear prominently.
- Regions with slowly varying intensities are **deemphasized**, making smooth areas appear darker.
- The resulting image typically consists of **grayish edge lines and discontinuities**, all set against a **dark, featureless background**, emphasizing important structural details.



Original
Image



Laplacian
Filtered Image



Sharpened
Image

Mid Term 2024

What are sharpening spatial filters? Discuss the response of first and second order derivatives. What is the Laplacian mask for image sharpening? 3

End Term 2024

How does 1st order derivative help in edge detection? Where can we use 2nd order derivative in edge detection? 3

Convolution and correlation:

Difference Between Convolution and Correlation:

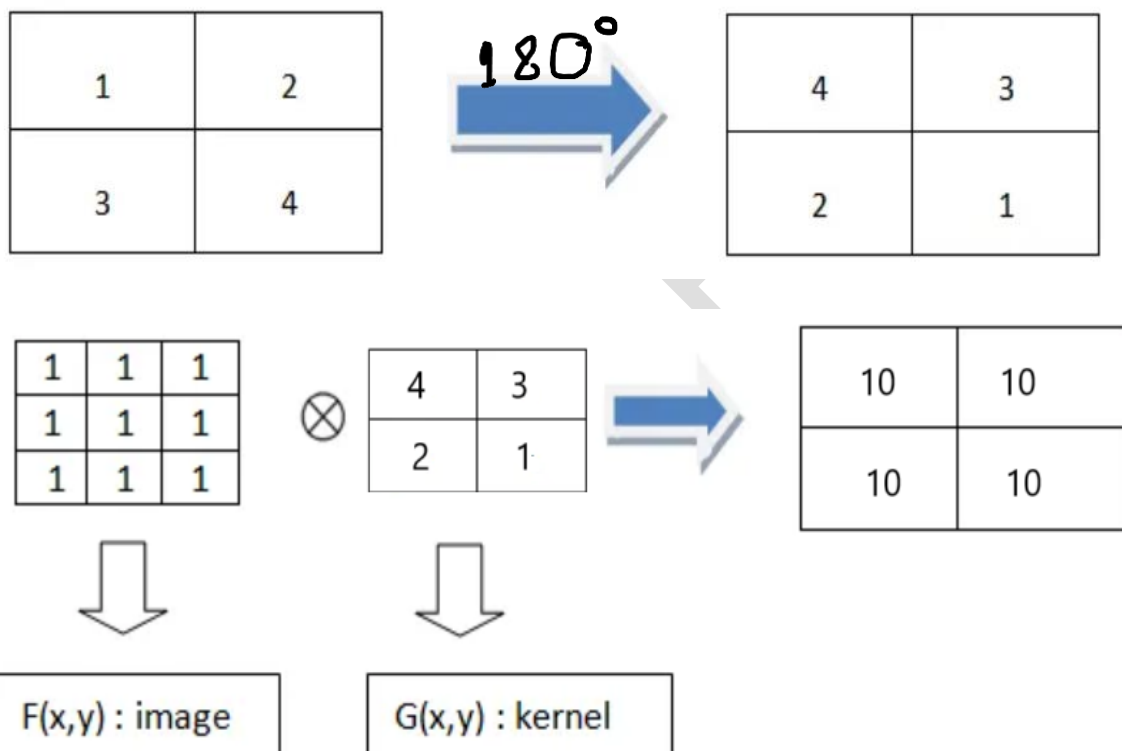
Aspect	Convolution	Correlation
Definition	A mathematical operation that combines two functions by flipping one and shifting it over the other.	Measures the similarity between two functions without flipping.
Formula	$(f * g)(x) = \sum f(m)g(x - m)$	$(f \circ g)(x) = \sum f(m)g(x + m)$
Flipping	The kernel (filter) is flipped before applying.	The kernel is applied directly without flipping.
Purpose	Used for filtering, edge detection, and feature extraction.	Used for pattern matching and similarity measurement.
Application	Common in image processing, convolutional neural networks (CNNs), and signal processing.	Used in template matching, signal analysis, and statistical correlation.
Effect	Sensitive to shifts and captures directional features.	Measures similarity but does not emphasize directional changes.

Key Difference:

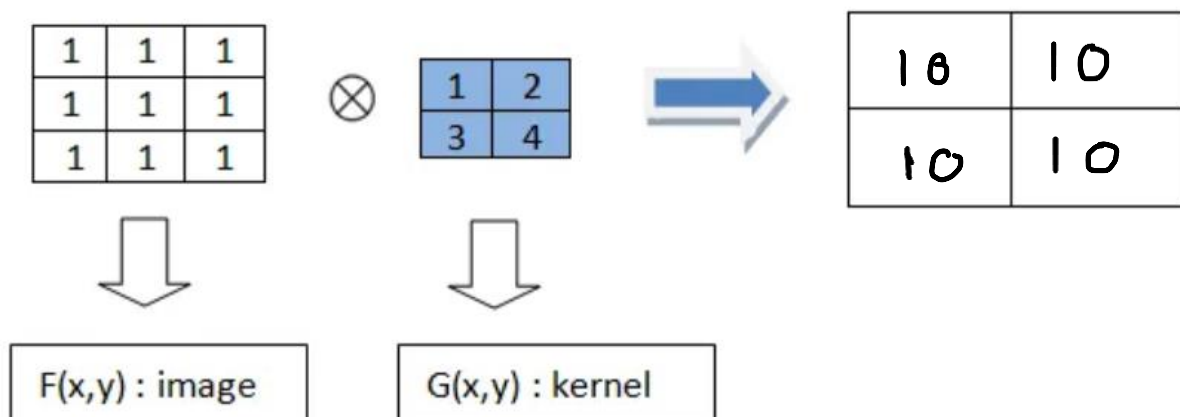
- **Convolution flips the filter (kernel) before applying it, while correlation does not.**
 - In **image processing**, convolution is used for blurring, sharpening, and edge detection, whereas correlation is often used for **template matching** and **feature detection**.
 - In the formulas for **convolution** and **correlation**, the symbols represent the following:
 - **$f(\mathbf{m})$** : The input function or signal (e.g., an image or a time-series signal).
 - **$g(\mathbf{x})$** : The filter (kernel) or another function used for processing.
 - **$\mathbf{x}$** : The position (index) where the operation is evaluated.
 - **$\mathbf{m}$** : A running variable (summation index) used to iterate over the function values.
 - In the convolution formula, the filter $g(\mathbf{m})$ is **flipped** before applying it and then shifted over $f(\mathbf{m})$.
 - In correlation formula, the filter $g(\mathbf{m})$ is applied **directly** without flipping and shifted over $f(\mathbf{m})$.
1. The convolution filtering is also a linear filtering and it is more common than correlation filtering. (In Linear filtering, **Slide the center of the kernel on the image. Multiply each weight in the kernel by the pixel in the**

image, sum these products, and replace them in the image).

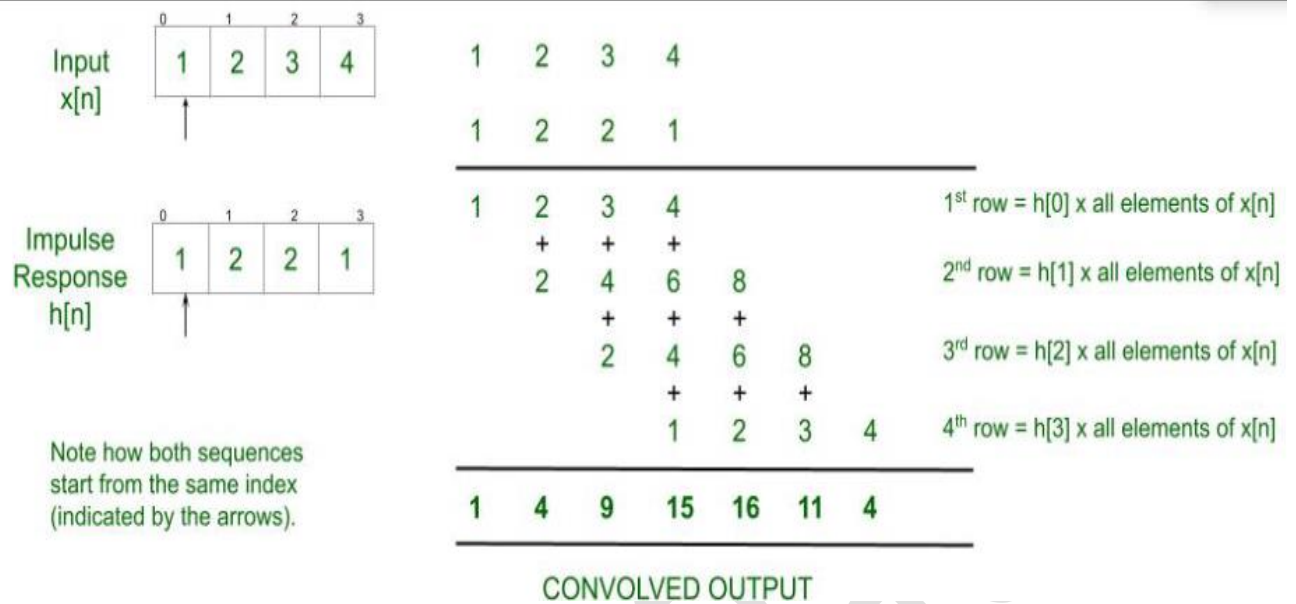
2. In convolution, Flip the filter in both directions (bottom to top, right to left).
3. Eg. $G(x,y)$ is a kernel. Convolve this kernel with the image $F(x,y)$. At first flip the kernel at 180 degrees.



4. In correlation filtering, Kernel (G) remains as it is.



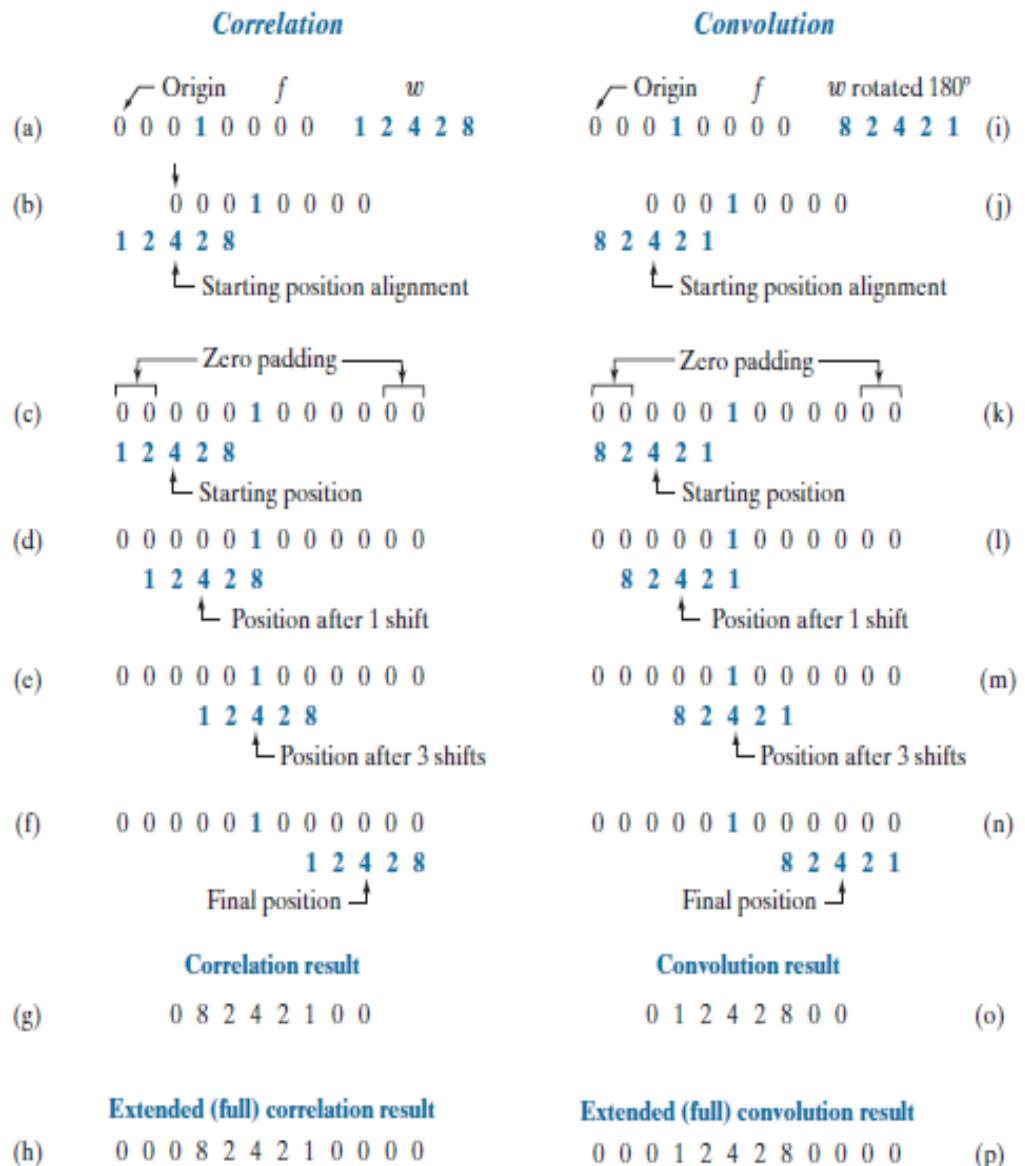
Linear convolution between two sequences:



Convolution and Correlation filtering:

FIGURE 3.29

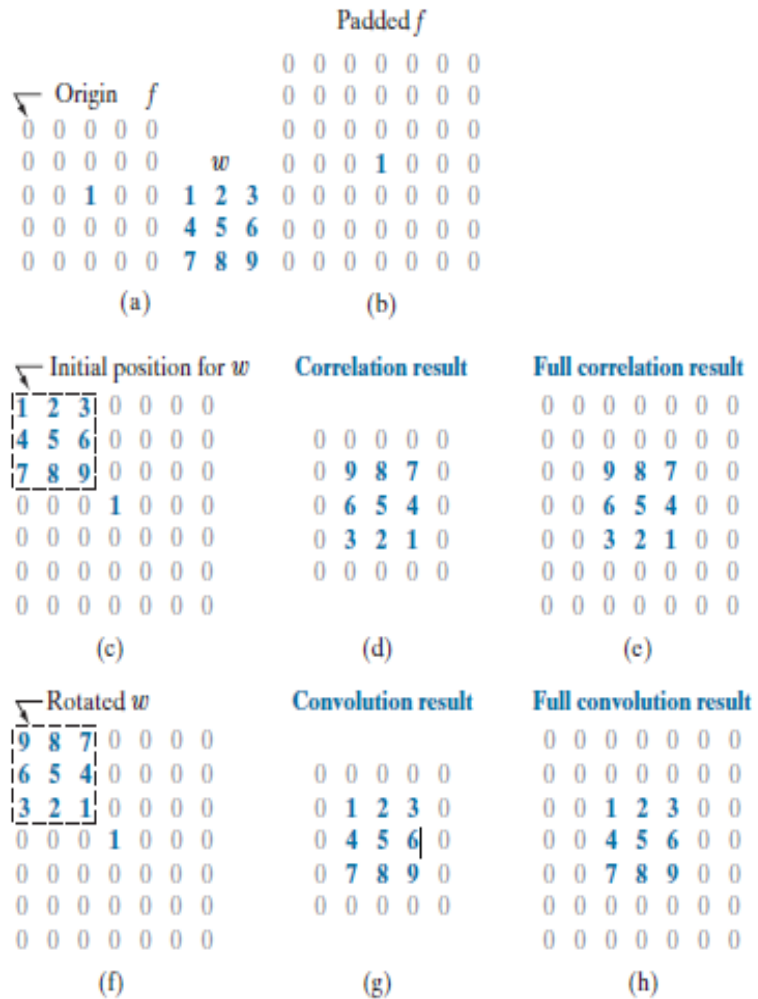
Illustration of 1-D correlation and convolution of a kernel, w , with a function f consisting of a discrete unit impulse. Note that correlation and convolution are functions of the variable x , which acts to *displace* one function with respect to the other. For the extended correlation and convolution results, the starting configuration places the right-most element of the kernel to be coincident with the origin of f . Additional padding must be used.



In the above example, f is a signal and w is a filter.

FIGURE 3.30

Correlation (middle row) and convolution (last row) of a 2-D kernel with an image consisting of a discrete unit impulse. The 0's are shown in gray to simplify visual analysis. Note that correlation and convolution are functions of x and y . As these variable change, they *displace* one function with respect to the other. See the discussion of Eqs. (3-36) and (3-37) regarding full correlation and convolution.



End Term 2024

A filter function is given by the sequence $x(n) = \{1, 2, 1\}$. Find the sequence $y(n)$ obtained by two convolution operation as shown below.

$$y(n) = (x(n) * (x(n) * x(n)))$$

Ans:

$$y(n) = (x(n) * \underbrace{(x(n) * x(n))}_a)$$

$$a = x(n) * x(n) \\ = \{1, 2, 1\} * \{1, 2, 1\}$$

$$\begin{array}{r} 1 2 1 \\ x 1 2 1 \\ \hline 1 2 1 \\ 2 4 2 \\ + 1 2 1 \\ \hline 1 4 6 4 1 \end{array}$$

$$y(n) = x(n) * a \\ = \{1, 2, 1, 0, 0\} * \{1, 4, 6, 4, 1\}$$

$$\begin{array}{r} 2 1 0 0 \\ x 1 4 6 4 1 \\ \hline 1 2 1 0 0 \\ 4 8 4 0 0 \\ 6 12 6 0 0 \\ 4 8 4 0 0 \\ 1 2 1 0 0 \\ + 2 1 0 0 \\ \hline 1 6 15 20 15 6 1 0 0 \end{array}$$

$$y(n) = \{1, 6, 15, 20, 15, 6, 1, 0, 0\}$$

[illegible]

- The L_4 Laplacian filter kernel:
$$\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

This filter calculates the Laplacian (second derivative) of each pixel, detecting edges.

Padding the Image: Since convolution requires examining the neighbors of every pixel, we pad the image with the border values. Padding with the border values means replicating the values of the outermost pixels.

The original image padded with border values becomes an 8x8 matrix:

$$\begin{bmatrix} 100 & 100 & 100 & 100 & 200 & 200 & 200 & 200 \\ 100 & 100 & 100 & 100 & 200 & 200 & 200 & 200 \\ 100 & 100 & 100 & 100 & 200 & 200 & 200 & 200 \\ 100 & 100 & 100 & 100 & 200 & 200 & 200 & 200 \\ 100 & 100 & 100 & 100 & 200 & 200 & 200 & 200 \\ 100 & 100 & 100 & 100 & 200 & 200 & 200 & 200 \\ 100 & 100 & 100 & 100 & 200 & 200 & 200 & 200 \\ 100 & 100 & 100 & 100 & 200 & 200 & 200 & 200 \end{bmatrix}$$

Convolution Process:

The convolution is done by sliding the 3x3 Laplacian filter over the image (excluding the padded border). For each pixel in the original image, the following steps occur:

Multiply the 3x3 neighborhood of the image by the Laplacian filter.

- Sum the results to compute the new value for the pixel.

Applying the Filter: Pixel-by-Pixel Example:

The pixel at (0,0) in the original image: 100

The 3x3 region around it (in the padded image) is:

$$\begin{bmatrix} 100 & 100 & 100 \\ 100 & \boxed{100} & 100 \\ 100 & 100 & 100 \end{bmatrix}$$

Applying the Laplacian filter:

$$\begin{aligned} & 0 \times 100 + 1 \times 100 + 0 \times 100 + 1 \times 100 + (-4 \times 100) + 1 \times \\ & 100 + 0 \times 100 + 1 \times 100 + 0 \times 100 \\ & = 100 + 100 - 400 + 100 + 100 = 0 \end{aligned}$$

So, the result at (0,0) is 0. Similarly calculate for the pixel at (0, 1).

The pixel at (0, 2) in the original image: 100

$$\begin{bmatrix} 100 & 100 & 200 \\ 100 & \boxed{100} & 200 \\ 100 & 100 & 200 \end{bmatrix}$$

The 3x3 region around it:

Applying the Laplacian filter:

$$\begin{aligned} & 0 \times 100 + 1 \times 100 + 0 \times 200 + 1 \times 100 + -4 \times 100 + 1 \times 200 \\ & + 0 \times 100 + 1 \times 100 + 0 \times 200 = 100 + 100 - 400 + 200 + 100 \\ & = 100 \end{aligned}$$

So, the result at (0,2) is 100.

The pixel at (0,3) in the original image: 200

The 3x3 region around it:
$$\begin{bmatrix} 100 & 200 & 200 \\ 100 & \boxed{200} & 200 \\ 100 & 200 & 200 \end{bmatrix}$$

Applying the Laplacian filter:

$$\begin{aligned} &0 \times 100 + 1 \times 200 + 0 \times 200 + 1 \times 100 + -4 \times 200 + 1 \times 200 \\ &+ 0 \times 100 + 1 \times 200 + 0 \times 200 = 200 + 100 - 800 + 200 + \\ &200 = -100 \end{aligned}$$

So, the result at (0,3) is -100.

Complete Convolution for other pixels of the original image

Applying this process to all pixels in the original image gives the final output:

$$\begin{bmatrix} 0 & 0 & 100 & -100 & 0 & 0 \\ 0 & 0 & 100 & -100 & 0 & 0 \\ 0 & 0 & 100 & -100 & 0 & 0 \\ 0 & 0 & 100 & -100 & 0 & 0 \\ 0 & 0 & 100 & -100 & 0 & 0 \\ 0 & 0 & 100 & -100 & 0 & 0 \end{bmatrix}$$

Mid Term 2024

- a. Consider the following image strip of 19 pixels.
Compute its First order derivative.

6	6	6	6	5	4	3	2	1	1	1	1	1	1	6	6	6	6	6
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Your answer should be in a tabular format having 2 rows as shown below. First row should show original signal; second row should have values of the First Order Derivative.

6	6	6	6	5	4	3	2	1	1	1	1	1	1	6	6	6	6	6

3

3. Solution: (a) Given strip:

6 6 6 6 5 4 3 2 1 1 1 1 1 1 6 6 6 6

The first-order derivative can be computed as the difference between consecutive pixel values.

First Order Derivative (difference between consecutive pixels):

$$6 - 6 = 0$$

$$6 - 6 = 0$$

$$6 - 6 = 0$$

$$6 - 5 = 1$$

$$5 - 4 = 1$$

$$4 - 3 = 1$$

$$3 - 2 = 1$$

$$2 - 1 = 1$$

$$1 - 1 = 0$$

$$1 - 1 = 0$$

$$1 - 1 = 0$$

$$1 - 1 = 0$$

$$1 - 1 = 0$$

$$1 - 6 = -5$$

$$6 - 6 = 0$$

$$6 - 6 = 0$$

$$6 - 6 = 0$$

$$6 - 6 = 0$$

So, the first-order derivative row is:

0 0 0 1 1 1 1 1 0 0 0 0 0 -5 0 0 0 0

b. Compute the convolution of the Laplacian kernel L_8 with the image given below. Use border values to extend the image.

100	100	10	10	10
100	100	10	10	10
100	100	10	10	10
100	100	10	10	10
100	100	10	10	10

Solution: (b) Compute the Convolution of the Laplacian Kernel L_8 : The kernel is not provided in the image, but a common Laplacian kernel (8-connected) looks like this:

$$\mathbf{L_8} = \begin{bmatrix} 1 & 1 & 1 \\ 1 & -8 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Given Image: $\begin{bmatrix} 100 & 100 & 10 & 10 & 10 \\ 100 & 100 & 10 & 10 & 10 \\ 100 & 100 & 10 & 10 & 10 \\ 100 & 100 & 10 & 10 & 10 \\ 100 & 100 & 10 & 10 & 10 \end{bmatrix}$

Since the question asks us to use border values to extend the image, we will assume that the values at the borders are replicated (i.e., the values at the edge are repeated outside the boundary). For example, the value 100 at the top-left corner would be extended beyond the image boundary.

Convolution Calculation:

Convolution at (0,0) (i.e., pixel 100 at top-left corner):

The neighborhood around (0,0) with border extension is:

$$\begin{bmatrix} 100 & 100 & 100 \\ 100 & 100 & 100 \\ 100 & 100 & 100 \end{bmatrix}$$

Apply the Laplacian kernel ($\mathbf{L_8}$):

$$\begin{aligned} &= (1 * 100) + (1 * 100) + (1 * 100) + (1 * 100) + (-8 * 100) \\ &+ (1 * 100) + (1 * 100) + (1 * 100) + (1 * 100) \\ &= 100 + 100 + 100 + 100 - 800 + 100 + 100 + 100 + 100 \end{aligned}$$

$$= 0$$

Convolution at (0,2) (i.e., pixel 10 in the middle):

$$\begin{aligned} & \text{The neighborhood around (0, 2) is: } \begin{bmatrix} 100 & 10 & 10 \\ 100 & 10 & 10 \\ 100 & 10 & 10 \end{bmatrix} \\ & = (1 * 100) + (1 * 10) + (1 * 10) + (1 * 100) + (-8 * 10) + \\ & (1 * 10) + (1 * 100) + (1 * 10) + (1 * 10) \\ & = 100 + 10 + 10 + 100 - 80 + 10 + 100 + 10 + 10 = 270 \end{aligned}$$

Convolution at (0,4) (i.e., pixel 10 in the last row and last column):

The neighborhood around (0,4) with border extension is:

$$\begin{aligned} & \begin{bmatrix} 10 & 10 & 10 \\ 10 & 10 & 10 \\ 10 & 10 & 10 \end{bmatrix} \\ & = (1 * 10) + (1 * 10) + (1 * 10) + (1 * 10) + (-8 * 10) + (1 \\ & * 10) + (1 * 10) + (1 * 10) + (1 * 10) \\ & = 10 + 10 + 10 + 10 - 80 + 10 + 10 + 10 + 10 = 0 \end{aligned}$$

After applying this process to all the pixels in the image.

The full convolution matrix will look like this:

$$\begin{bmatrix} 0 & -270 & 270 & 0 & 0 \\ 0 & -270 & 270 & 0 & 0 \\ 0 & -270 & 270 & 0 & 0 \\ 0 & -270 & 270 & 0 & 0 \\ 0 & -270 & 270 & 0 & 0 \end{bmatrix}$$

a. You have a 4 x 5 image as follows:

2	3	5	5	6
5	3	9	11	12
12	11	1	8	1
7	7	9	0	1

5

Perform gray level slicing with respect to given function – clearly mention the new pixel values in a 4 x 5 matrix. Also

represent the function graphically.

$$s = \begin{cases} 15 & \text{if } 5 < r \leq 10 \\ r & \text{otherwise} \end{cases}$$

Solution: To solve this gray-level slicing problem, we apply the given transformation function to each pixel in the 4x5 image. The function is defined as:

$$s = 15, \text{ if } 5 < r \leq 10$$

$$= r, \text{ otherwise}$$

This means that if the pixel value (r) lies between 6 and 10 (both inclusive), we replace it with 15. Otherwise, we keep the pixel value the same.

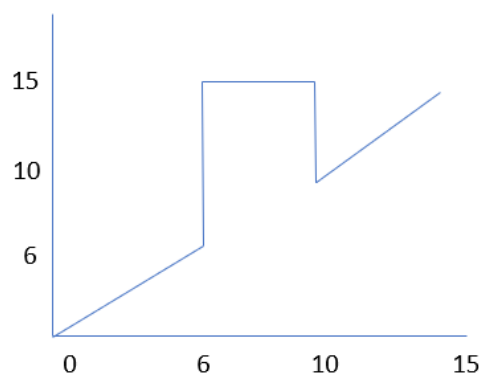
Original 4x5 Image:
$$\begin{bmatrix} 2 & 3 & 5 & 5 & 6 \\ 5 & 3 & 9 & 11 & 12 \\ 12 & 11 & 8 & 1 & 1 \\ 7 & 7 & 9 & 0 & 1 \end{bmatrix}$$
; r is each intensity value in the matrix.

Applying the transformation: if $5 < r \leq 10$, replace r with 15, otherwise, keep the value of r .

Transformed Image:
$$\begin{bmatrix} 2 & 3 & 5 & 5 & 15 \\ 5 & 3 & 15 & 11 & 12 \\ 12 & 11 & 15 & 1 & 1 \\ 15 & 15 & 15 & 0 & 1 \end{bmatrix}$$

Graphical Representation of the Function:

The function is a step function. You can represent it as



follows:

- For $r \leq 5$, $s = r$ (the graph is a linear line with slope 1).

- For $5 < r \leq 10$, ($s = 15$) (a flat line at ($s = 15$)).
- For $r > 10$, ($s = r$) (the graph continues linearly with slope 1).
- The plot would look like a line that steps up to 15 for the values between 6 and 10 and returns to the original value after 10.

Perform Gray Level slicing on the following 3 bit image. Let $A = 3$ & $B = 5$. Give the modified image with and without background transformations in the format given below. Also plot the graph for both types of transformations.

5

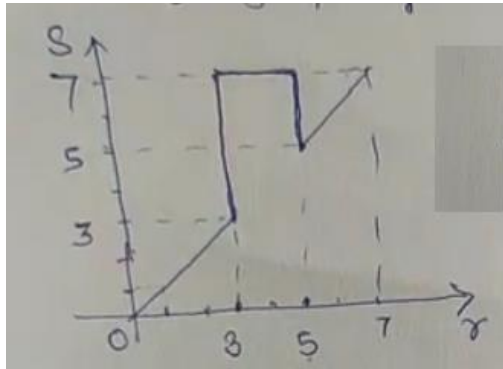
2	2	1	1	2
2	3	4	5	2
6	7	6	7	2
2	6	5	6	1
1	0	3	3	1

Modified image without
Background

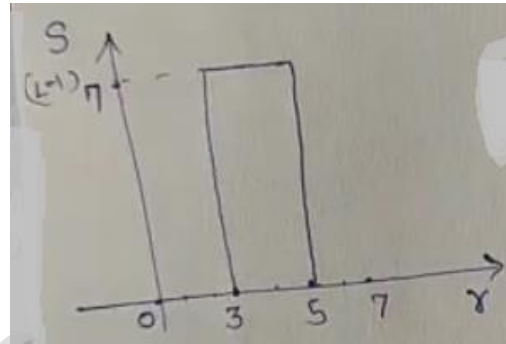
Modified image with
Background

. Solution: (a) For Gray level slicing, $A = 3$, $B = 5$. For 3-bit, $L = 2^3 = 8$. (Given)

For with Background



For without Background



$$S = (L-1); A \leq r \leq B$$

$$= r; \quad \text{Otherwise}$$

$$\begin{bmatrix} 2 & 2 & 1 & 1 & 2 \\ 2 & 7 & 7 & 7 & 2 \\ 6 & 7 & 6 & 7 & 2 \\ 2 & 6 & 7 & 6 & 1 \\ 1 & 0 & 7 & 7 & 1 \end{bmatrix}$$

$$S = (L-1); A \leq r \leq B$$

$$= 0; \quad \text{Otherwise}$$

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 7 & 7 & 7 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 7 & 0 & 0 \\ 0 & 0 & 7 & 7 & 0 \end{bmatrix}$$

b. Which of the following is true for Power Law Transformation? Write the complete true statement in your answer sheet.

- $\gamma > 1$: Compresses dark values; Expands bright values
- $\gamma > 1$: Expands dark values; Compresses bright values
- $\gamma < 1$: Compresses dark values; Expands bright values
- $\gamma = 1$: Expands dark values; Compresses bright values

Solution: (b): Power Law Transformation

The correct statement for Power Law Transformation is:

- $\text{Gamma} < 1$: Expands dark values and compresses bright values. This means it lightens dark areas and reduces the intensity of bright areas. This is typically used for enhancing details in darker regions of an image.
- $\text{Gamma} > 1$: Compresses dark values and expands bright values. This makes dark areas darker and bright areas brighter, emphasizing high-intensity regions and reducing details in darker parts.

Visual Effect:

- $\text{Gamma} < 1$: Makes the image overall brighter, emphasizing lower intensity (darker) regions.
- $\text{Gamma} > 1$: Makes the image darker, emphasizing higher intensity (brighter) regions.

So, for $\text{gamma} < 1$, dark areas are lightened, and bright areas are compressed (diminished in intensity).

- b. Find the values of logical operation XOR and set operation $A - B$ for the binary images A and B. Assume 1 to be foreground pixels.

0	0	0	0	0	0	0	0
0	0	0	1	1	1	1	0
0	0	0	0	1	1	1	0
0	0	0	0	1	1	1	0
0	0	0	0	0	0	0	0

A

0	0	0	0	0	0	0	0
0	0	1	1	1	0	0	0
0	0	1	1	1	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

B

Solution: (b) To solve the problem of finding the XOR and set operation ($A - B$) for the given binary images (A) and (B)

XOR Operation

The XOR operation between two binary matrices is performed element-wise, where the result is 1 if the elements are different and 0 if they are the same.

Calculation of XOR:

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Set Operation (A - B):

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

The set operation (A – B) is performed element-wise, where the result is 1 if the element in (A) is 1 and the corresponding element in (B) is 0, otherwise the result is 0.

In two separate 7 x 7 grids, mark all the pixels which are at a distance of ≤ 3 from pixel located at (3, 3), when the distance measure is (i) City Block (ii) Chess Board

Part b: Distance Measures in a 7x7 Grid

City Block Distance (Manhattan Distance)

The City Block Distance between two points (x₁, y₁) and (x₂, y₂) is calculated as:

$$d_CityBlock = |x_1 - x_2| + |y_1 - y_2|$$

For this part, we need to mark all pixels within a distance of (≤ 3) from the pixel located at (3, 3) on a 7x7 grid.

Chessboard Distance (Chebyshev Distance)

The Chessboard Distance between two points (x_1, y_1) and (x_2, y_2) is calculated as:

$$d_Chessboard = \max(|x_1 - x_2|, |y_1 - y_2|)$$

We will also mark all pixels within a distance of (≤ 3) from the same pixel using this distance measure.

Here are the marked grids for both distance measures:

1. City Block Distance:

Pixels within a distance (≤ 3) from the pixel (3,3) (center) are marked as 1:

$$\begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

2. Chessboard Distance:

Pixels within a distance (≤ 3) from the pixel (3,3) (center) are marked as 1:

$$\begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

In the Chessboard distance, all pixels within a radius of 3 are covered, while the City Block distance has a diamond-shaped region.

Assume that you are given grey scale image with pixel intensities between [37 ... 209]. What point operation is necessary to map the pixels to the [0 ... 255] range?

... a ... image of size 100 x

3

Solution: Mapping pixel intensities between [37 ... 209] to [0 ... 255]. This is a linear transformation problem where the original pixel intensity range needs to be mapped to a new range i.e. contrast stretching. The formula is

$$s = s_{\min} + \frac{(r - r_{\min}) \cdot (s_{\max} - s_{\min})}{r_{\max} - r_{\min}}$$

Where:

r is the pixel value in the original range.

$r_{\min}=37$, $r_{\max}=209$, $s_{\max}=255$, $s_{\min}=0$.

Thus, the formula becomes:
$$S = \frac{(r - 37)255}{172}$$

This point operation will linearly scale the pixel intensities from the original range [37, 209] to [0, 255].

If $X = \{2 \ 3 \ 4 \ 3 \ 4 \ 5 \ 6\}$ and $w = \{1 \ 1 \ 1\}$ what will be the result of Median filtering? Use Zero padding if required.

Solution: (c) The input sequence is $X = [2, 3, 4, 3, 4, 5, 6]$, and the median filter window is $w = [1, 1, 1]$. This is a 3-point median filter, meaning we take the median of each window of 3 consecutive values.

To apply zero padding at the boundaries:

The padded sequence becomes: $[0, 2, 3, 4, 3, 4, 5, 6, 0]$.

Now, apply the median filter over windows of size 3:

1. median (0, 2, 3) = 2
2. median (2, 3, 4) = 3
3. median (3, 4, 3) = 3
4. median (4, 3, 4) = 4
5. median (3, 4, 5) = 4
6. median (4, 5, 6) = 5
7. median (5, 6, 0) = 5

The result after median filtering is: $[2, 3, 3, 4, 4, 5, 5]$

b. Suppose there is a multispectral image of size 100×100 . This image has 4 bands and each band is using 256 gray levels. Find the size of the image in bits.

3

Solution: (b) The image has the following characteristics:

Dimensions: 100×100 pixels.

Number of bands: 4.

Each band has 256 gray levels, meaning each pixel in each band uses 8 bits (since $256 = 2^8$).

The size of the image in bits is calculated by:

The number of pixels $\times$ The number of bands $\times$ The number of bits per pixel in each band

Total bits = $100 \times 100 \times 4 \times 8 = 320,000$ bits

End Term 2023-2024

- a. One mobile phone has higher spatial resolution while another has higher gray level resolution. Which mobile would you prefer to buy?
- b. What will be the size in bits of an RGB image using 3 bits for each channel and having 1024 rows and 512 columns?
- c. How many different 10×10 binary images can exist?
- a. An 8-bit digital image has gray levels in the range from 0 to 255. If image negation transform is applied, then in which range will the pixels lie?
- e. Consider the image shown below. Assume

4

that this is a standard grayscale image, with pixel values in the range $[0, 255]$. What can you say about the contrast of the image? Give your answer in one sentence.

10	20	30	40	50	60	70
255	20	30	40	255	60	70
10	20	30	255	50	60	70
10	255	30	40	50	60	255
10	20	30	40	50	60	70

End Term 2023

a. Consider the following 5 bit, 4 x 4 image.

	0	1	2	3
0	31	30	29	28
1	16	26	25	24
2	24	25	26	16
3	28	29	30	31

- i. Extract the 1st Bit Plane. [2]
- ii. How will the image look if thresholding is set at 27? The output image should still be a 5 bit image. [1]
- iii. What will be the new value of the pixel (2, 2) if smoothing is done using a 3 x 3 average filter? [1]

4

End Term 2023

Consider the following two images – (a) & (b). | 3



(a)



(b)

Certain logical operators were applied on them to obtain the following results.



(c)



(d)

Choose the correct option from the following.
Write the complete correct option in your answer sheet.

- Image (c) is the result of $a(x,y) \wedge b(x,y)$ and (d) is the result of $a(x,y) \vee b(x,y)$
- Image (c) is the result of $a(x,y) \wedge b(x,y)$ and (d) is the result of $a(x,y) \wedge (\sim b(x,y))$
- Image (c) is the result of $a(x,y) \vee b(x,y)$ and (d) is the result of $a(x,y) \wedge (\sim b(x,y))$

End Term 2024

A common measure of transmission for digital data is the baud rate, defined as the number of bits transmitted per second. Transmission is accomplished in packets consisting of a start bit, a byte (8 bits) of information and a stop bit.

a) How many minutes would it take transmit a 1024×1024 image with 256 gray levels if we use a 56 k baud modem?

b) What would be the time required if we use a 850 k baud transmission line?

Perform histogram stretching so that the new image has a dynamic range of $[0, 7]$.

Gray level	0	1	2	3	4	5	6	7
No. of pixels	0	0	50	60	20	10	40	0

Suppose that a 3-bit image ($L=8$) of size 10×10 pixels ($MN = 100$) has the intensity distribution shown in the following table A. Get the histogram transformation function and make the output image with the specified histogram, listed in the table B.

Histogram A

Gray Level	0	1	2	3	4	5	6	7
No. of Pixels	4	17	15	18	24	12	0	10

Histogram B

Gray Level	0	1	2	3	4	5	6	7
No. of Pixels	0	0	0	36	24	12	8	20

End Term 2023

In the 8x8 image X shown below, all pixels shown with a * have gray level 41, all the other pixels have gray level 14. We linearly smooth X using a 1-row by 3-column averaging mask whose origin is the center of the mask. How many of the 64 pixels in X will change value? What is the largest increase in brightness any pixel in the given X will have? Use similar value padding for boundary pixels.

	*				*		
	*				*		
	*	*		*	*		
	*	*	*	*	*		
		*	*	*			
		*	*	*			
			*				
			*				

4

Give your answer in the following format: [3]

Row no.	No. of pixels whose intensity will change
1 (Top row)	
2	
3	
4	
5	
6	
7	
8 (Last Row)	

b. We have an image with 3 bit / pixel with the following histogram.

Gray-level value	0	1	2	3	4	5	6	7
No of Pixels	10	8	9	2	13	1	5	2

Perform histogram equalization. Show all the steps.

End Term 2022

Define image resolution. What are spatial resolution and intensity resolution? What kind of resolution do you prefer in your mobile phone screen?	4
How contrast stretching helps in image enhancement? Define any three intensity transformation functions.	4
How convolution and correlation is used for linear spatial filtering? Perform each of them on 1-D function $f=0\ 0\ 0\ 1\ 0\ 0\ 0\ 0$, and a filter $w=1\ 2\ 3\ 2\ 8$	4
What are Sharpening Spatial Filters? Discuss the response of First and Second order derivatives. What is the Laplacian mask for image sharpening?	3

What is Digital Image Processing? Explain fundamental steps in Digital Image Processing.	4
Perform histogram equalization of an image shown below: $f(m,n) = \begin{bmatrix} 3 & 2 & 4 & 5 \\ 7 & 7 & 8 & 2 \\ 3 & 1 & 2 & 3 \\ 5 & 4 & 6 & 7 \end{bmatrix}$	4

Assume a 4-bit gray scale image I as shown below, generate the linear contrast stretched image with minimum gray level 5 and maximum gray level 14.

3	3	3	3	3	3	3	3
3	4	4	4	4	4	4	3
3	4	2	2	2	2	4	3
3	4	2	5	5	2	4	3
3	4	2	5	5	2	4	3
3	4	2	2	2	2	4	3
3	4	4	4	4	4	4	3
3	3	3	3	3	3	3	3

4

Assume a 5-bit gray scale image I and consider the spatial filter H given by

$$H = \begin{bmatrix} -1 & -2 & 0 \\ -2 & 0 & 3 \\ 0 & 3 & 1 \end{bmatrix}$$

Determine the maximum and minimum possible values that a pixel, to which this spatial filter is applied, can have. Do not apply any type of normalization.